Tecnología Orientada a Objetos

Unidad 1. Introducción al Paradigma

¿Qué es un Paradigma?

¿Qué es un Paradigma?

Una forma de pensar

Un paradigma de programación es una manera o estilo de programación de software. Existen diferentes formas de diseñar un lenguaje de programación y varios modos de trabajar para obtener los resultados que necesitan los programadores. Se trata de un conjunto de métodos sistemáticos aplicables en todos los niveles del diseño de programas para resolver problemas computacionales.



El Paradigma Imperativo (El enfoque tradicional)

Los programas consisten en una sucesión de instrucciones o conjunto de sentencias, como si el programador diera órdenes concretas. El desarrollador describe en el código paso por paso todo lo que hará su programa

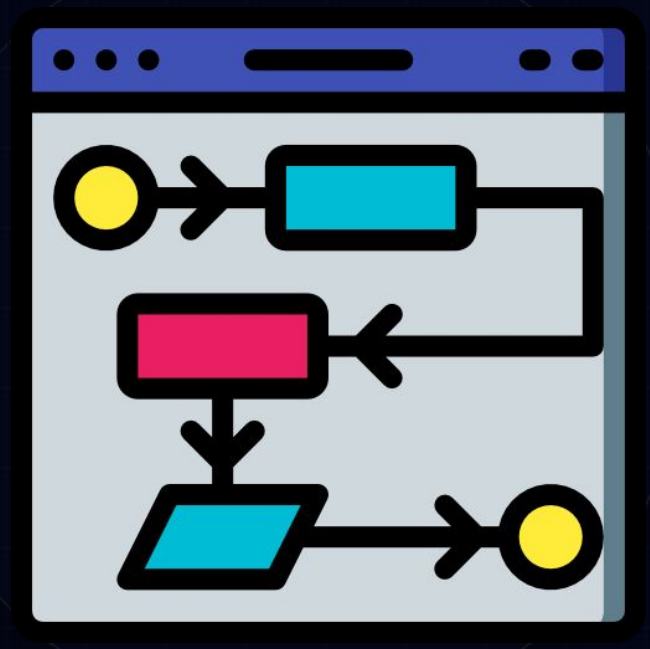
Lenguajes Representativos: C, C++, Pascal, COBOL.

Enfoques subordinados:

Estructurada: Control de flujo mediante bucles y condicionales.

Procedimental: Agrupa expresiones repetitivas en funciones o procedimientos reutilizables.

Modular: Divide el programa en módulos independientes para facilitar su manejo y lectura en problemas complejos.



¿Por qué necesitamos la POO?

Los programas reales son grandes y complejos.

- Necesitamos organizar arquitecturas complejas para evitar el "código espagueti".
- Requerimos reutilizar código de forma eficiente y segura en múltiples módulos.
- Buscamos modelar problemas del mundo real de una manera lógica e intuitiva.



Concepto Central: Objeto

Una instancia que representa entidades del mundo real.

Todo objeto posee dos características fundamentales:



1. Datos (Atributos)

Describen el **estado** del objeto. Son variables internas. Por ejemplo, para un objeto "Persona", sus atributos serían el nombre, la edad y la estatura.



2. Comportamiento (Métodos)

Describen las **funciones o acciones** que el objeto puede realizar. Modifican el estado interno. Para el objeto "Persona", sus métodos podrían ser caminar, hablar o comer.

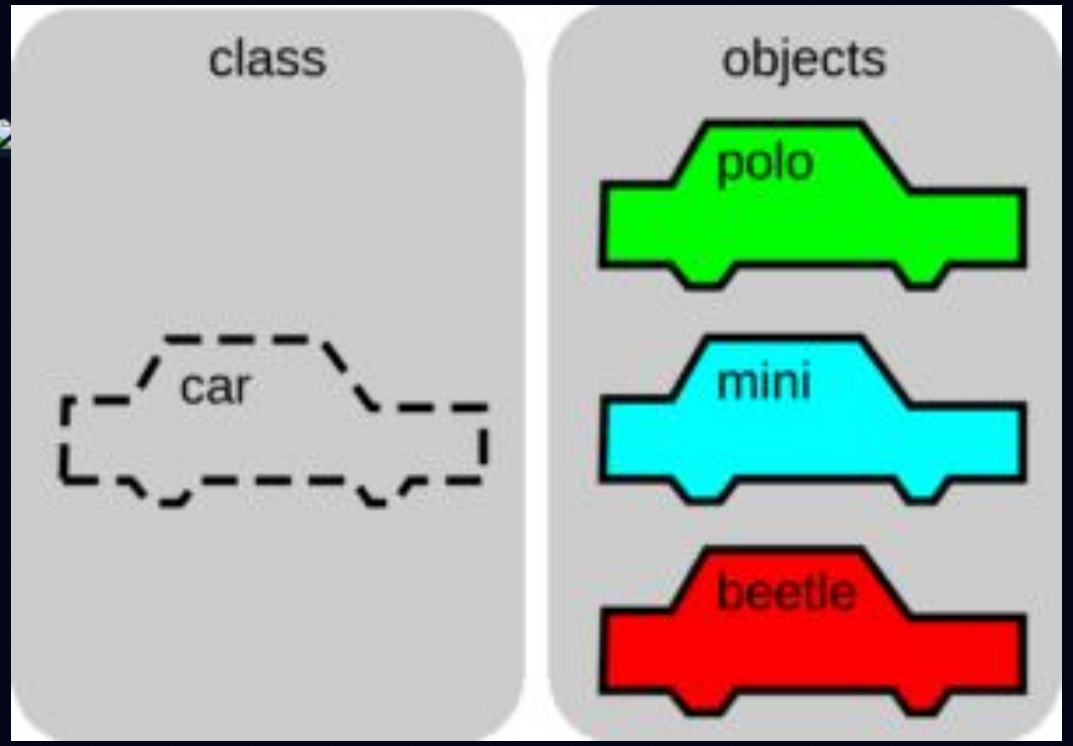
Concepto Central: Clase

El Molde o Plantilla

Una clase es el plano arquitectónico a partir del cual se construyen o "instancian" los objetos.

- Define **qué atributos** tendrá el objeto.
- Define **qué métodos** podrá ejecutar.

> *Ejemplo:*
> Clase: `Persona`
> Atributos definidos: `nombre`, `edad`
> Métodos definidos: `hablar()`

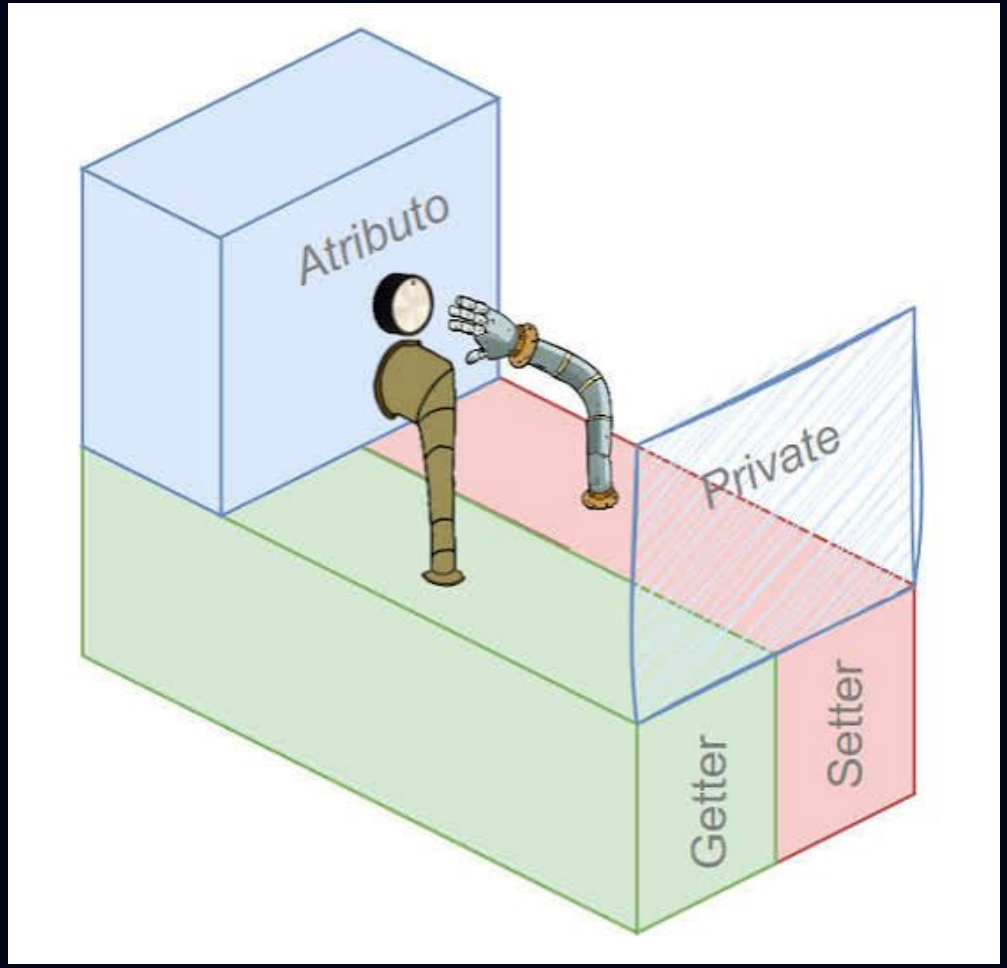



POO
POO
Encapsulation
Inhertance
Polymporism
Absraction
Los 4 Pilares
POO en Java

1. Encapsulamiento

La encapsulación contiene toda la información importante de un objeto dentro del mismo y solo expone la información seleccionada al mundo exterior.
Esta propiedad permite asegurar que la información de un objeto esté oculta para el mundo exterior, agrupando en una Clase las características o atributos que cuentan con un acceso privado, y los comportamientos o métodos que presentan un acceso público.

La encapsulación de cada objeto es responsable de su propia información y de su propio estado.

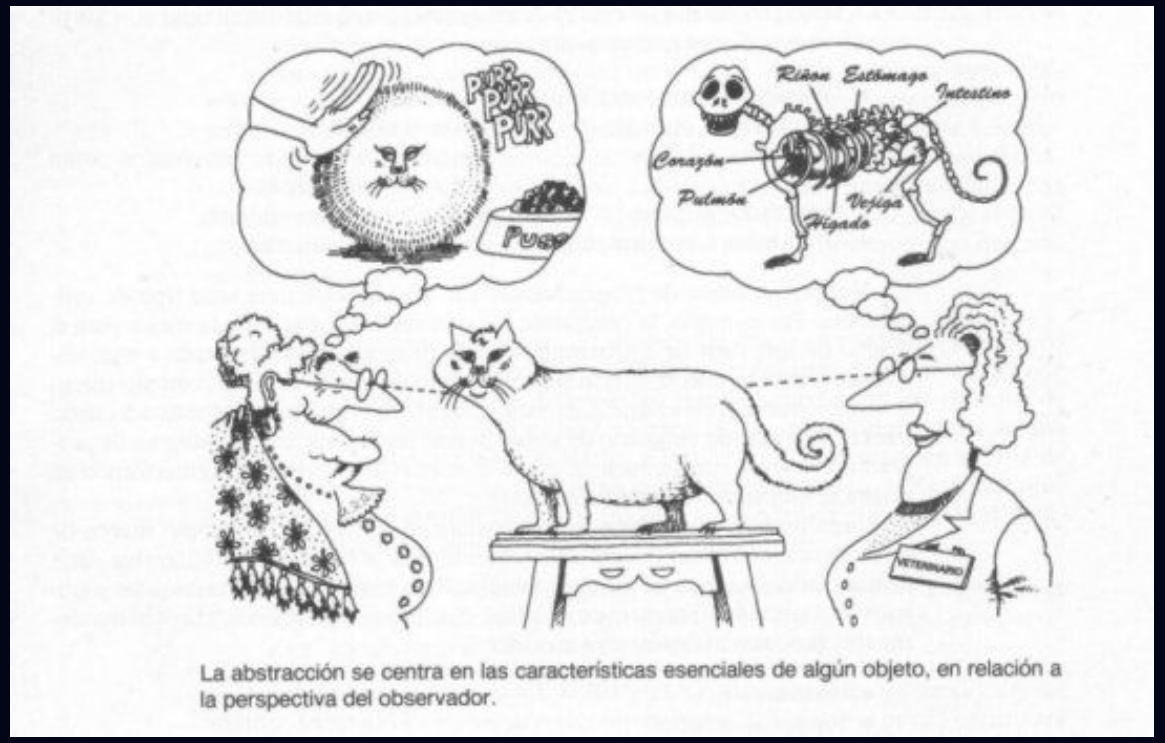


2. Abstracción

La abstracción es cuando el usuario interactúa solo con los atributos y métodos seleccionados de un objeto, utilizando herramientas simplificadas de alto nivel para acceder a un objeto complejo.

La abstracción se basa en usar cosas simples para representar la complejidad. Los objetos y las clases representan código subyacente, ocultando los detalles complejos al usuario. Por consiguiente, supone una extensión de la encapsulación.

Siguiendo con el ejemplo del coche, no es necesario que conozcas todos los detalles de cómo funciona el motor para poder conducirlo.

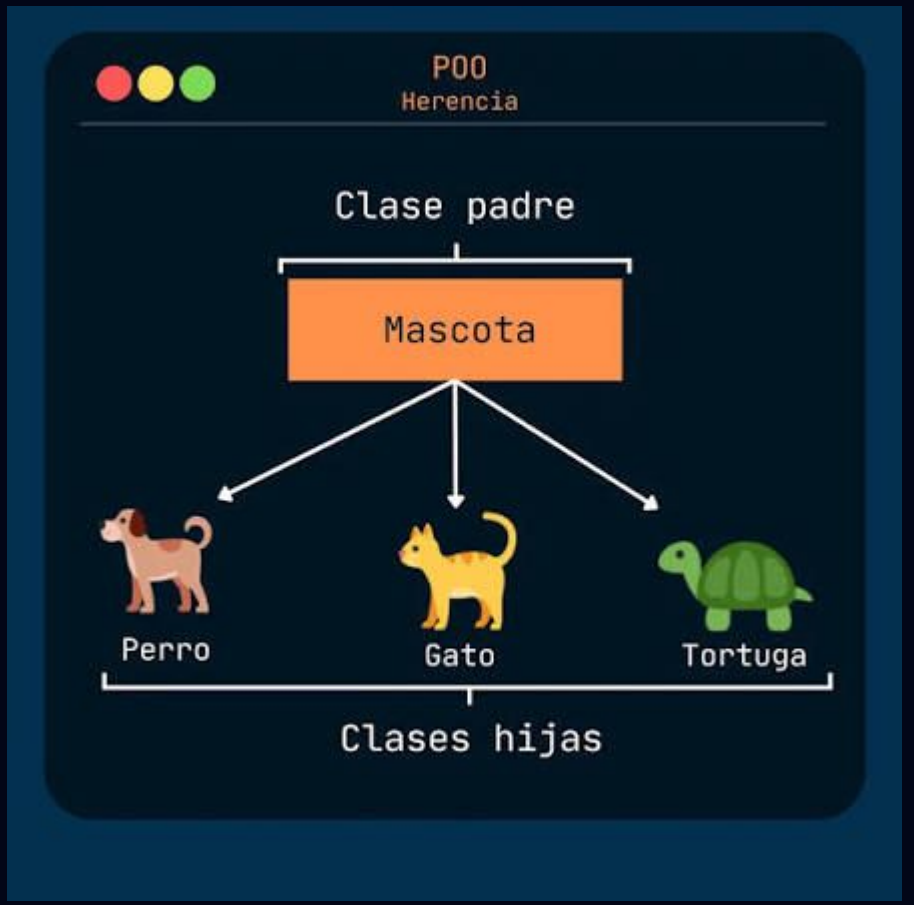


La abstracción se centra en las características esenciales de algún objeto, en relación a la perspectiva del observador.

3. Herencia

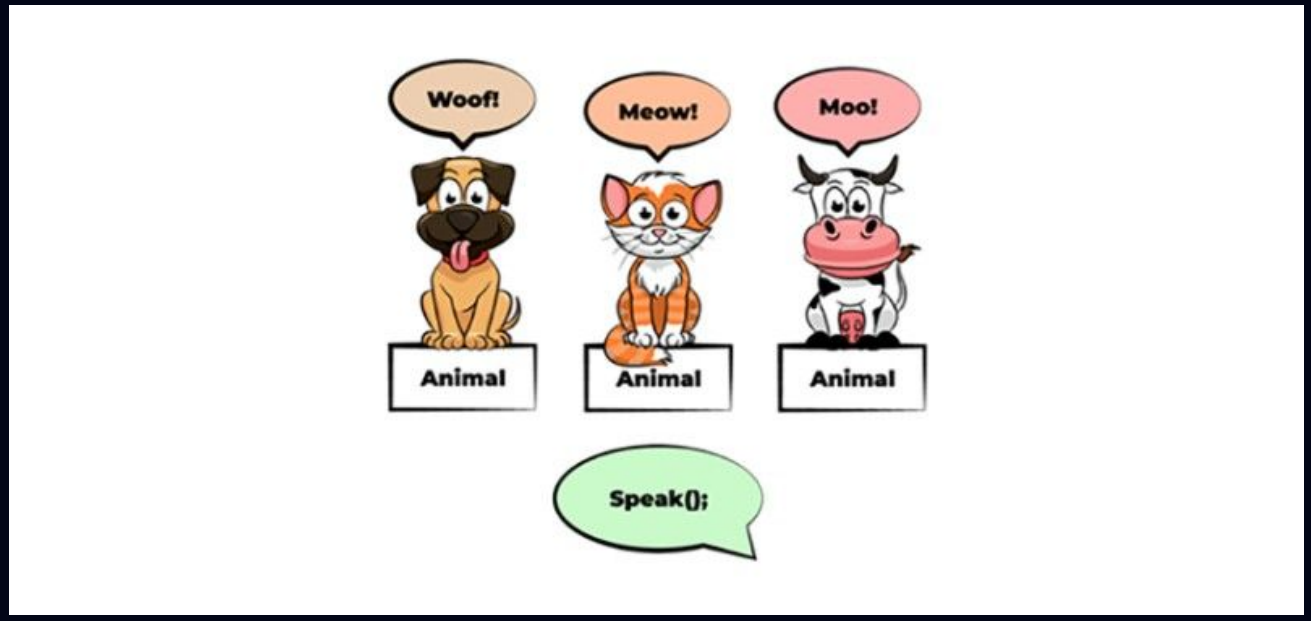
La herencia define relaciones jerárquicas entre clases, de forma que atributos y métodos comunes puedan ser reutilizados. Las clases principales extienden atributos y comportamientos a las clases secundarias.

Volviendo al ejemplo de los animales, se puede usar una sola clase de animal y agregar un atributo de tipo de animal que especifique el tipo de animal. Los diferentes tipos de animales necesitarán diferentes métodos, por ejemplo, las aves deben poder poner huevos y los peces, nadan. Incluso cuando los animales tienen un método en común, como moverse, la implementación necesitaría muchas declaraciones «si» para garantizar el comportamiento de movimiento correcto. Por ejemplo, las ranas saltan, mientras que las serpientes se deslizan. El principio de herencia nos permite solucionar este problema.



4. Polimorfismo

El polimorfismo consiste en diseñar objetos para compartir comportamientos, lo que nos permite procesar objetos de diferentes maneras. Es la capacidad de presentar la misma interfaz para diferentes formas subyacentes o tipos de datos. Al utilizar la herencia, los objetos pueden anular los comportamientos principales compartidos, con comportamientos secundarios específicos. El polimorfismo permite que el mismo método ejecute diferentes comportamientos de dos formas: anulación de método y sobrecarga de método.



Beneficios de POO

- **Reutilización del código.**
- Convierte cosas complejas en **estructuras simples reproducibles.**
- **Evita la duplicación** de código.
- Permite **trabajar en equipo** gracias al encapsulamiento ya que minimiza la posibilidad de duplicar funciones cuando varias personas trabajan sobre un mismo objeto al mismo tiempo.
- Al estar la clase bien estructurada permite **la corrección de errores** en varios lugares del código.
- **Protege la información** a través de la encapsulación, ya que solo se puede acceder a los datos del objeto a través de propiedades y métodos privados.
- La abstracción nos permite construir sistemas más complejos y de una forma más **sencilla y organizada**.

¿Dudas o Preguntas?

Siguiente clase: Creación e Interacción de Objetos.